Step-by-Step Guide: Build an End-to-End AWS Web Application

A hands-on walkthrough for building a "Power of Math" web app with **AWS Amplify, API Gateway, Lambda, DynamoDB and IAM**. Based on the video *AWS Project: Architect and Build an End-to-End AWS Web Application from Scratch*.

1. What You Will Build

A web page where a user enters two numbers (a base and an exponent). The page calls a REST API, a Lambda function calculates the result, stores it in DynamoDB and returns it to the page.

Request flow: Browser (Amplify-hosted page) → API Gateway → Lambda → DynamoDB. IAM supplies the permissions Lambda needs to write to DynamoDB.

The four things the app needs:

- A way to create and host a webpage (**Amplify**)
- A way to invoke the math functionality (**API Gateway**)
- A way to do the math (**Lambda**)
- Somewhere to store and return the result (**DynamoDB**, with **IAM** permissions)

2. Prerequisites

- A text editor (Notepad, Notepad++, VS Code)
- An AWS account with access to the Management Console
- Basic knowledge of AWS
- Use one AWS Region throughout (for example, the Region closest to you) so all services can find each other

3. Step 1: Create the Web Page

1. Create a folder on your computer, for example `power-of-math`.
2. In your text editor, create a file named `index.html` and paste the starter page below.

3. Save it, then compress the file (or the folder contents) into a `.zip` archive. `index.html` must be at the top level of the zip.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>To the Power of Math!</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 40px; }
    label, input, button { font-size: 16px; margin: 6px 0; }
  </style>
  <script>
    // Replace with your API Gateway Invoke URL later (Step 8)
    var API_ENDPOINT = "YOUR-API-INVOKE-URL";

    var callAPI = (base, exponent) => {
      var myHeaders = new Headers();
      myHeaders.append("Content-Type", "application/json");
      var raw = JSON.stringify({ "base": base, "exponent": exponent });
      var requestOptions = { method: 'POST', headers: myHeaders, body: raw,
redirect: 'follow' };
      fetch(API_ENDPOINT, requestOptions)
        .then(response => response.text())
        .then(result => alert(JSON.parse(result).body))
        .catch(error => console.log('error', error));
    }
  </script>
</head>
<body>
  <h1>To the Power of Math!</h1>
  <form>
    <label>Base number:</label>
    <input type="number" id="base"><br>
    <label>... to the power of:</label>
    <input type="number" id="exponent"><br>
    <button type="button"
onclick="callAPI(document.getElementById('base').value,
document.getElementById('exponent').value)">CALCULATE</button>
  </form>
</body>
</html>
```

4. Step 2: Deploy and Host with AWS Amplify

1. In the AWS Console, search for **AWS Amplify** and open it.
2. Choose **Create new app**, then pick the option to deploy **without a Git provider** (manual deploy).
3. Enter an app name (for example `PowerOfMath`) and a branch name (for example `dev`).
4. Choose **Drag and drop**, upload your `.zip` file, and click **Save and deploy**.
5. When deployment finishes, open the **domain URL** Amplify shows. You should see your page.

The page does nothing yet, because there is no backend.

5. Step 3: Create the Lambda Function (the Math)

1. Open **AWS Lambda** and click **Create function**.
2. Choose **Author from scratch**.
3. Name it `PowerOfMathFunction`, runtime **Python 3.x**, and leave the default execution role option (a new basic role is created).
4. Click **Create function**.
5. In the code editor, replace the contents of `lambda_function.py` with:

```
import json
import math

def lambda_handler(event, context):
    base = int(event['base'])
    exponent = int(event['exponent'])
    result = math.pow(base, exponent)
    return {
        'statusCode': 200,
        'body': json.dumps('Your result is ' + str(result))
    }
```

6. Click **Deploy**.

Test the function

1. Open the **Test** tab and create a new test event named `test`.
2. Use this JSON:

```
{
  "base": 2,
  "exponent": 3
}
```

3. Click **Test**. The response should contain `Your result is 8.0`.

6. Step 4: Create a REST API with API Gateway

1. Open **Amazon API Gateway** and choose **Create API**.
2. Select **REST API** (not private) and click **Build**.
3. Name it `PowerOfMathAPI` and create it.
4. Choose **Create resource**. Name it `power-of-math` and **tick Enable CORS**.
5. Select the resource and choose **Create method**.
   - Method type: **POST**
   - Integration type: **Lambda function**
   - Choose your `PowerOfMathFunction` (same Region)
6. Click **Create method**.
7. Select the resource again and, if CORS is not already enabled, choose **Enable CORS** and allow the POST method.
8. Click **Deploy API**. Create a new stage called `dev`.
9. Copy the **Invoke URL** shown for the stage. Your endpoint is that URL followed by `/power-of-math`.

Optional: use the **Test** feature on the POST method with a body of `{"base": 2, "exponent": 3}` to confirm it works.

7. Step 5: Create the DynamoDB Table

1. Open **Amazon DynamoDB** and choose **Create table**.
2. Table name: `PowerOfMathDatabase`.
3. Partition key: `ID` (type **String**).
4. Leave the other settings at their defaults and click **Create table**.

8. Step 6: Give Lambda Permission to Write to DynamoDB

Lambda runs under an **execution role**. By default it cannot touch DynamoDB, so you must add a permission.

1. In your Lambda function, open **Configuration → Permissions** and click the **role name** (this opens IAM).
2. Choose **Add permissions → Create inline policy**.
3. Switch to the **JSON** tab and paste the policy below, replacing the Region and account ID with your own:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["dynamodb:PutItem", "dynamodb:GetItem"],
      "Resource": "arn:aws:dynamodb:YOUR-REGION:YOUR-ACCOUNT-
ID:table/PowerOfMathDatabase"
    }
  ]
}
```

4. Name the policy `PowerOfMathDynamoPolicy` and click **Create policy**.

Tip: you can copy the table's exact ARN from the DynamoDB table's **Additional info** section. Granting only the actions and table you need follows the principle of least privilege.

9. Step 7: Update the Lambda Code to Save Results

Replace the function code with the version below, then click **Deploy**.

```
import json
import math
import boto3
from time import gmtime, strftime

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('PowerOfMathDatabase')

def lambda_handler(event, context):
```

```
    now = strftime("%a, %d %b %Y %H:%M:%S +0000", gmtime())
    base = int(event['base'])
    exponent = int(event['exponent'])
    result = math.pow(base, exponent)

    table.put_item(Item={
        'ID': str(result),
        'LatestGreetingTime': now
    })

    return {
        'statusCode': 200,
        'body': json.dumps('Your result is ' + str(result))
    }
```

Run the same `test` event again. Then open DynamoDB, choose your table, and click **Explore table items**. You should see a new item with the result.

10. Step 8: Connect the Web Page to API Gateway

1. Open `index.html` in your text editor.
2. Replace `YOUR-API-INVOKE-URL` with your full endpoint, for example `https://abc123.execute-api.us-east-1.amazonaws.com/dev/power-of-math`.
3. Save the file and re-create the `.zip` archive.

11. Step 9: Redeploy with Amplify

1. Return to your app in **AWS Amplify**.
2. Choose **Deploy updates** and drag and drop the new `.zip`.
3. Wait for the deployment to complete and open the domain URL.

12. Step 10: Test the Finished Application

1. Enter a base (for example 3) and an exponent (for example 4).
2. Click **CALCULATE**. A pop-up should say `Your result is 81.0`.
3. Check DynamoDB to confirm the new item was saved.

13. Troubleshooting

- **Nothing happens, or a CORS error in the browser console:** enable CORS on the API Gateway resource, then **redeploy the API** to your stage.
- **"Missing Authentication Token":** the URL is missing the resource path or stage name.
- **Internal server error (500):** open Lambda's **Monitor → View CloudWatch logs**. An `AccessDeniedException` means the IAM policy is wrong or the table name or Region does not match.
- **Old page still showing:** hard-refresh the browser after redeploying in Amplify.
- **Changes to API Gateway not applied:** API Gateway changes only take effect after you click **Deploy API**.

14. IMPORTANT: Clean Up Your Resources

Delete everything when you are done so you do not incur charges.

1. **Amplify:** delete the app.
2. **API Gateway:** delete the `PowerOfMathAPI` API.
3. **Lambda:** delete `PowerOfMathFunction`.
4. **DynamoDB:** delete the `PowerOfMathDatabase` table.
5. **IAM:** delete the Lambda execution role (and its inline policy) that was created for the function.
6. **CloudWatch:** delete the log group `/aws/lambda/PowerOfMathFunction`.

Appendix: Video Timestamps

- 01:07 Overview of services and application
- 01:56 Architecting the web application
- 02:28 What you need to follow along
- 03:02 Creating and hosting the web page
- 04:27 Deploying with AWS Amplify
- 06:56 Creating the Python Lambda function
- 08:44 Testing the Lambda function
- 10:54 Creating the REST API in API Gateway
- 15:44 Creating the DynamoDB table

- 16:37 Giving Lambda permission to write to DynamoDB
- 18:23 Updating the Lambda code
- 21:08 Updating index.html to call API Gateway
- 23:03 Redeploying with Amplify
- 24:32 Deleting your resources